

LLM-Driven Navigation Agent in a Webots 3D World

Diksha Jangam

May 31, 2026

Contents

1	Introduction	2
2	System Overview	2
2.1	High-level pipeline	2
2.2	World and objects	2
3	Design Choices	2
3.1	LLM multi-path reasoning	2
3.2	A* path planning on an occupancy grid	3
3.3	Waypoint-based controller	3
3.4	Discrete action space	3
3.5	CSV logging	3
3.6	Visualization	4
4	Implementation Details	4
4.1	LLM configuration	4
4.2	Occupancy grid and A*	4
4.3	Goal stop radius	4
4.4	Waypoint following	5
4.5	Goal completion detection	5
4.6	CSV logging implementation	5
4.7	Visualization implementation	5
5	Running the System	5
5.1	Dependencies	5
5.2	Webots setup	6
5.3	Environment variable	6
5.4	Changing the route	6
5.5	Outputs	6
6	Example Logs and Behaviour	6
6.1	Console output	6
6.2	CSV snippet	6
7	Discussion and Limitations	7
7.1	Strengths	7
7.2	Limitations	7
8	Conclusion	7

1 Introduction

This document describes a complete intelligent agent system that navigates a 3D Webots environment using a combination of:

- Large Language Model (LLM) reasoning for goal selection,
- A* path planning on an occupancy grid,
- A waypoint-following controller,
- Automatic visualization of the world and path,
- CSV logging of the agent's behaviour.

The agent receives a natural-language instruction such as:

“Go to the yellow goal, then the red goal, then the green goal.”

It then interprets this instruction, selects an ordered sequence of goals, plans a collision-free path, and executes it in Webots.

2 System Overview

2.1 High-level pipeline

The system follows this pipeline:

1. **User instruction** is defined in the controller code.
2. **LLM multi-path reasoning** generates several candidate goal sequences.
3. A **judge LLM** scores and selects the best candidate.
4. An **occupancy grid** is built from static obstacles.
5. **A* path planning** computes a path for each goal segment.
6. A **waypoint controller** drives the robot along the path.
7. **CSV logging** records behaviour; **visualization** is generated at the end.

2.2 World and objects

The Webots world contains:

- A differential-drive robot with GPS and IMU,
- Three named goals (red, green, yellow) with fixed (x, y) coordinates,
- Rectangular static obstacles.

The goals are stored as:

```
OBJECTS = [  
  {"name": "red goal",    "x": 0.19,  "y": 1.24},  
  {"name": "green goal",  "x": 1.15,  "y": -1.26},  
  {"name": "yellow goal", "x": -2.25, "y": 0.48}  
]
```

3 Design Choices

3.1 LLM multi-path reasoning

Instead of a single LLM call, the system:

- Generates multiple candidate goal sequences using different style constraints (e.g., shortest distance, clockwise order, safety).

- Uses a separate “judge” LLM to score each candidate on:
 - Faithfulness to the user instruction,
 - Route reasonableness,
 - Efficiency,
 - Clarity of coordinates.
- Selects the candidate with the highest score.

This improves robustness and reduces the chance of a single poor LLM output.

3.2 A* path planning on an occupancy grid

A grid-based occupancy map is built around the robot:

- Obstacles are inflated by a safety margin to avoid tight passages.
- Each grid cell is marked as free or occupied.
- A* search is used to find a path between start and goal cells.

A* is chosen for its determinism, simplicity, and suitability for static grid environments.

3.3 Waypoint-based controller

The continuous path is discretized into waypoints. For each waypoint:

- The robot computes distance and heading error using GPS and IMU,
- A simple rule-based controller chooses one of:
 - GO_FORWARD,
 - TURN_LEFT,
 - TURN_RIGHT,
 - STOP.

A speed profile reduces forward speed near the goal to improve stability.

3.4 Discrete action space

The action space is intentionally kept discrete:

- Easier to reason about and debug,
- Stable for simple navigation tasks,
- Sufficient to demonstrate LLM-driven planning and control.

3.5 CSV logging

The system logs each control step to a CSV file:

- Robot position (x, y) ,
- Yaw (heading),
- Waypoint index,
- Action,
- Distance to waypoint,
- Heading error.

This satisfies the requirement for “example inputs and outputs” and enables offline analysis.

3.6 Visualization

A Matplotlib-based visualization is generated at the end of the run:

- Obstacles as gray rectangles,
- Start position,
- Goals with correct colors (red, green, yellow),
- Selected path as a polyline.

The figure is saved as `map.png` in the controller folder.

4 Implementation Details

4.1 LLM configuration

The system uses a Groq-hosted LLM via an OpenAI-compatible API:

- API key is read from the `OPENAI_API_KEY` environment variable,
- Two models are used:
 - Main model for candidate generation,
 - Judge model for scoring candidates.

The multi-path generation function:

```
best_goals, meta = solve_goals_with_multipath(  
    user_instruction, OBJECTS, n_candidates=4  
)
```

returns a list of goal coordinates and metadata about candidates and scores.

4.2 Occupancy grid and A*

The grid resolution and radius are defined as:

```
GRID_RES    = 0.20  
GRID_RADIUS = 10
```

Obstacles are inflated by a margin:

```
OBSTACLE_INFLATION = 0.15
```

The planner:

- Converts world coordinates to grid indices,
- Runs A* to find a sequence of grid cells,
- Converts the grid path back to world coordinates.

4.3 Goal stop radius

To avoid colliding with the goal object, the robot stops at a configurable radius before the exact goal:

```
GOAL_STOP_RADIUS = 0.30
```

The final target for planning is a “safe” point slightly before the goal along the line from the current position.

4.4 Waypoint following

At each control step:

1. The current waypoint (x_t, y_t) is selected.
2. Distance and heading error are computed from GPS and IMU.
3. If distance is below a threshold, the waypoint is marked as reached.
4. Otherwise, an action is chosen based on heading error and distance.

4.5 Goal completion detection

The system tracks which waypoints correspond to the end of each goal segment. When the robot reaches the final waypoint for a goal, it prints:

```
[GOAL] <goal name> reached successfully.
```

Goal names are recovered by matching the LLM-generated goal coordinates with the original OBJECTS list.

4.6 CSV logging implementation

A CSV file `log.csv` is created in the project root. Each step appends a row:

```
x, y, yaw, waypoint_index,  
action, distance_to_waypoint, heading_error
```

This file can be used to:

- Reconstruct trajectories,
- Analyze controller behaviour,
- Provide evidence of the agent acting in the world.

4.7 Visualization implementation

At the end of the run, a function:

```
visualize_world_and_paths(...)
```

is called. It:

- Draws obstacles as rectangles,
- Plots goals with color-coded markers:
 - red goal → red marker,
 - green goal → green marker,
 - yellow goal → yellow marker,
- Plots the selected path as a polyline,
- Saves the figure as `map.png`.

5 Running the System

5.1 Dependencies

Python dependencies:

```
pip install matplotlib requests
```

5.2 Webots setup

1. Open Webots.
2. Load the world file `plane_env.wbt`.
3. Set the robot controller to `controller_plane_env`.

5.3 Environment variable

Set the LLM API key:

```
export OPENAI_API_KEY=your_key_here    # Linux/macOS
set OPENAI_API_KEY=your_key_here       # Windows
```

5.4 Changing the route

The user instruction is defined in the controller code, e.g.:

```
user_instruction = "Go to the yellow goal, then the red goal, then the green goal"
```

To change the route, modify this line (e.g., line 430 in the controller file) with a new natural-language instruction.

5.5 Outputs

After running the simulation:

- `log.csv` in the project root contains the behaviour log,
- `map.png` in the controller folder contains the visualization,
- The Webots console shows:
 - LLM candidate scores,
 - Waypoint progress,
 - Goal completion messages.

6 Example Logs and Behaviour

6.1 Console output

Typical console output:

```
[LLM] Generating multi-path goal candidates...
[LLM] Winner index: 0
[PLANNER] Occupied cells: 60
[SYSTEM] Waypoint 14 reached.
[GOAL] red goal reached successfully.
[SYSTEM] Waypoint 27 reached.
[GOAL] green goal reached successfully.
[SYSTEM] All waypoints completed.
Saved visualization to: controllers/controller_plane_env/map.png
```

6.2 CSV snippet

An example row in `log.csv`:

```
-1.80, -0.70, 1.57, 12, GO_FORWARD, 0.35, 0.10
```

7 Discussion and Limitations

7.1 Strengths

- Clear separation between high-level reasoning (LLM) and low-level control,
- Deterministic planning via A*,
- Transparent behaviour via CSV logging and visualization,
- Natural-language route specification.

7.2 Limitations

- Static obstacles only; no dynamic obstacle handling,
- Simple discrete controller (no advanced control theory),
- Relies on LLM quality and prompt design.

8 Conclusion

This system demonstrates a complete intelligent-agent pipeline in a Webots 3D world:

- LLM-based goal interpretation,
- Multi-path candidate generation and judging,
- A* path planning on an occupancy grid,
- Waypoint following with a discrete action controller,
- Behaviour logging and visualization.

It satisfies the challenge requirements for:

- A working agent in a virtual world,
- Clear observation and action formats,
- Example inputs and outputs (via logs),
- Documented design choices.